Lab program-13:

13. Construct a C program for implementation of the various memory allocation

Code:

```c
#include <stdio.h>

void firstFit(int blockSize[], int m, int processSize[], int n)
{
    int allocation[20];

    for (int i = 0; i < n; i++)
        allocation[i] = -1;

    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < m; j++)
        {
            if (blockSize[j] >= processSize[i])
            {
                allocation[i] = j;
                blockSize[j] -= processSize[i];
                break;
            }
        }
    }

    printf("\nProcess\tProcess Size\tBlock Allocated\n");

    for (int i = 0; i < n; i++)
    {
        printf("%d\t%d\t\t", i + 1, processSize[i]);

        if (allocation[i] != -1)
            printf("%d\n", allocation[i] + 1);
        else
            printf("Not Allocated\n");
    }
}

void bestFit(int blockSize[], int m, int processSize[], int n)
{
    int allocation[20];

    for (int i = 0; i < n; i++)
        allocation[i] = -1;
```

```
    for (int i = 0; i < n; i++)
    {
        int bestIndex = -1;

        for (int j = 0; j < m; j++)
        {
            if (blockSize[j] >= processSize[i])
            {
                if (bestIndex == -1 ||
                    blockSize[j] < blockSize[bestIndex])
                {
                    bestIndex = j;
                }
            }
        }

        if (bestIndex != -1)
        {
            allocation[i] = bestIndex;
            blockSize[bestIndex] -= processSize[i];
        }
    }

    printf("\nProcess\tProcess Size\tBlock Allocated\n");

    for (int i = 0; i < n; i++)
    {
        printf("%d\t%d\t\t", i + 1, processSize[i]);

        if (allocation[i] != -1)
            printf("%d\n", allocation[i] + 1);
        else
            printf("Not Allocated\n");
    }
}

void worstFit(int blockSize[], int m, int processSize[], int n)
{
    int allocation[20];

    for (int i = 0; i < n; i++)
        allocation[i] = -1;

    for (int i = 0; i < n; i++)
    {
        int worstIndex = -1;
```

```
        for (int j = 0; j < m; j++)
        {
            if (blockSize[j] >= processSize[i])
            {
                if (worstIndex == -1 ||
                    blockSize[j] > blockSize[worstIndex])
                {
                    worstIndex = j;
                }
            }
        }

        if (worstIndex != -1)
        {
            allocation[i] = worstIndex;
            blockSize[worstIndex] -= processSize[i];
        }
    }

    printf("\nProcess\tProcess Size\tBlock Allocated\n");

    for (int i = 0; i < n; i++)
    {
        printf("%d\t%d\t\t", i + 1, processSize[i]);

        if (allocation[i] != -1)
            printf("%d\n", allocation[i] + 1);
        else
            printf("Not Allocated\n");
    }
}

int main()
{
    int blockSize[20], processSize[20];
    int m, n, choice;

    printf("Enter number of memory blocks: ");
    scanf("%d", &m);

    printf("Enter size of each memory block:\n");

    for (int i = 0; i < m; i++)
        scanf("%d", &blockSize[i]);

    printf("Enter number of processes: ");
    scanf("%d", &n);
```

```
    printf("Enter size of each process:\n");

    for (int i = 0; i < n; i++)
        scanf("%d", &processSize[i]);

    printf("\n1. First Fit");
    printf("\n2. Best Fit");
    printf("\n3. Worst Fit");
    printf("\nEnter your choice: ");

    scanf("%d", &choice);

    switch (choice)
    {
        case 1:
            firstFit(blockSize, m, processSize, n);
            break;

        case 2:
            bestFit(blockSize, m, processSize, n);
            break;

        case 3:
            worstFit(blockSize, m, processSize, n);
            break;

        default:
            printf("Invalid Choice");
    }

    return 0;
}
```

OUTPUT:

```
PS C:\Users\SMILE\Documents\OS\Lab programs> ./lab_13.EXE
Enter number of memory blocks: 4
Enter size of each memory block:
2
3
4
3
Enter number of processes: 4
Enter size of each process:
2
5
4
3

1. First Fit
2. Best Fit
3. Worst Fit
Enter your choice: 1

Process Process Size    Block Allocated
1       2               1
2       5               Not Allocated
3       4               3
4       3               2
```